64KB → Config
High Mem MMIO

Clear



DATE : □□ - □□ - □□□□

## SNOR Bootflow

BootROM (Internal Silicon ROM)

Address: 0x FFFF0000 (64KB)

Built into ARM cortex R8 SOC

Hardcoded by Samsung during chip design

1. Minimal Hardware Initialization
   - Basic clock setup (minimal) → through SFR access
   - Pre-configured SPI Controller (hardwired)
   - Fixed SPI pins, SPI timings / speed
   - Fixed Snor addressing mode
   - No driver needed - hardware - level control.

2. Read Bootloader from predefined SNOR location
   - used hardwired SPI to read address 0x0
   - or config register (strap pins)
   - copy ~64KB bootloader to 0x2C00000 (SRAM)

3. Jump to Bootloader

   PC = 0x2C00000

   - Branch to · START UP - BOOTLOADER vector

DATE : ☐☐ - ☐☐ - ☐☐☐☐

Stage 1: Details (Boot ROM)

Boot ROM has pre-configured SPI Controller settings
Read activated slot no. from SNOR to determine which slot contains fw
Read dracofw.bin from that slot
Parse the header (0x41C for bootloader size)
Read bootloader binary starting from offset 0x800 copies to 0x2C00000 (64KB SRAM region allocated).
Jumps to 0x2C00000

Stage 2 (bootloader)

bootloader fully initializes SPI driver
Read entire fw image from SNOR to SRAM buffer
Uses GDMA to copy individual sections to their execution addresses.

DATE : □□ - □□ - □□□□

Bootloader Startup.asm.
Address 0x2c00000 (Shared SRAM)

- Reset Handler:
  - Stack Setup (6 modes)
  - Zero ITCM / DTCM.
  - Enable TCM ECC
  - BL -- Bootloader_Main().

Bootloader Main :: Main()
- Now Initialize full SPI driver
- Initialize GDMA
- Read Main Fw from SNOR (Slot 1-4)
- Decrypt main fw.
- Copy Main Fw to ITCM / DTCM via GDM
- Jump to 0x0 (main fw Reset Handl

Main Fw RTOS Startup.s
Address : 0x00000000 (ITCM)

Reset Handler:
PC Relocation
MPU Configuration
Clock Config
Branch to Main().

DATE :

Reset Handler:

$bootloader is a small SNOR to TCM loader that runs first, validates/copies the main fw image into place and then hands control over to main fw at ITCM address 0x0. which starts freeRTOS.

Page: ...../...../.....

Bootloader phase loads at 0x2C00000

Startup.o placed at very beginning.

Followed by .RO, .RW, .ZI sections

Startup- bootloader vector table

Reset Handler:

Initialize stack pointers for processor modes (SVC, IRQ, FIQ, ABT, UNDEF etc)

Zero Init ITCM and DTCM

Enable TCM ECC in Auxiliary Control Register

Clear Register R0-R12

Branch to -- Bootloader_Main

↙

Initialize Objects to run global CPP constructors

Calls Main()

↓

~~Initialize SPI~~

If bootInfo → downloadMode == true

~~Save firmware from SRAM buffer~~ to SNOR, then loops forever

Initialize GDMA

Reads active FW slots from SNOR into ~~firmware~~ shared SRAM

For each binary section in firmware header, copies it from buffer to its execution address via GDMA

Optionally runs DICE. Bootloader operation

High Mem MMIO

Date: ...../...../.....

Jumps to mainfw which is 0x0 ( start of ITCM/ mainFw R0 region)

Main FW phase → (loads/executes at ITCM 0x0)

RTOS startup.o at beginning.

All RTOS code placed in ITCM.

RW/ZI and stack/heap placed in DTCM

--freeR tos-vector-table at 0x0.

Reset Handler

Clear SCTLR.V bit so vectors are at 0x0

Disable mpu

Configure 9 MPU regions.

Enable dynamic clock gating

ReEnable MPU

Set stack pointers for SVC, IRQ, FIQ, ABT, UNDEF, SYS/USR mo

Zero Init SRAM-ZI

Clear Register R0-R12

Branch to --main ( ARM C library /runtime entry)

Multi, Watchdog, Core Timer, Peri Timer, Watchdog

Date: ...../...../....

Reset

↓

Boot ROM / Boot code

↓

Reset / Startup ASM

↓

Set SP — CPU setup — Vector table

MPU/cache/

clocks/etc

↓

Initialize .data (ROM to RAM copy)

↓

Zero initialize .bss

↓

C Runtime initialization

↓

~~Main~~

↓

firmware / RTOS init

application

FreeRTOS

↓

Scheduler start